

Recommendation & Customer Feedback

Rapport technique et metier

Pipeline SQL → NLP → Recommendation → Evaluation → Dashboard
applique a 200 000 avis Amazon (categorie All Beauty)

Projet de portfolio Data Science
Septembre 2026

1. Resume executif

Ce projet construit, de bout en bout, un systeme hybride de **recommandation produit** et d'**analyse des avis clients** pour un catalogue e-commerce reel (200 000 avis Amazon, categorie *All Beauty*, 60 726 produits, 168 659 utilisateurs). Il repond a deux besoins metier concrets a partir d'une seule source de donnees : personnaliser les recommandations (Marketing/Growth) et comprendre le sentiment et les themes recurrents dans les avis (Qualite/Produit).

Le systeme final est **hybride** : un moteur de collaborative filtering pour les clients avec un historique suffisant, complete par un moteur content-based pour les clients avec peu d'historique. Chaque brique a ete **evaluee quantitativement** (Precision@K / Recall@K) contre une baseline de popularite, et chaque choix methodologique important a fait l'objet d'une experimentation documentee plutot que d'un choix arbitraire.

Indicateur	Valeur
Avis traites	200 000 (199 119 apres nettoyage)
Produits catalogue	60 726
Utilisateurs uniques	168 659
Note moyenne	4,05 / 5
Themes NLP identifies	8 (NMF)
Collaborative filtering — lift Recall@20 vs popularite	+428,6% (606 utilisateurs evalues)
Content-based (fallback) — lift Recall@5 vs popularite	+326,2% (15 631 utilisateurs evalues)
Couverture combinee du systeme hybride	16 237 / 168 659 utilisateurs (9,6%)

Limite majeure. Les utilisateurs restants (148 429, soit 88% de la base) n'ont laisse qu'un seul avis : aucune methode basee sur l'historique ne peut les servir sans donnee supplementaire. C'est une limite honnete du projet, detaillee en section 11.

2. Contexte metier

Scenario : un e-commerçant reçoit des milliers d'avis clients (note + texte libre) sur son catalogue produit. Deux directions ont des besoins distincts mais liés :

- **Marketing / Growth** — augmenter le taux de conversion et la retention en proposant des recommandations personnalisées, plutôt que d'afficher les mêmes produits populaires à tout le monde.
- **Qualité / Produit** — comprendre le sentiment général et les thèmes récurrents (odeur, texture, livraison, prix...) par produit, sans avoir à lire manuellement des milliers d'avis.

Le choix structurant de ce projet est de répondre aux **deux besoins à partir de la même table d'avis brute** (notes + texte) : la partie structurée (note, identifiants utilisateur/produit) alimente le moteur de recommandation, la partie non structurée (texte libre) alimente le pipeline NLP. Cela démontre comment une seule source de données peut être exploitée de deux façons complémentaires plutôt que de traiter les deux problèmes indépendamment.

3. Source de donnees et dictionnaire des colonnes

Amazon Reviews 2023 (McAuley Lab, UCSD), categorie **All Beauty**. Echantillon de **200 000 avis** sur 701 500 disponibles dans la categorie complete.

3.1 Pourquoi cette source et cette categorie

- Contient **a la fois** des notes structurees (user_id, item_id, rating) et du texte libre (avis) — necessaire pour alimenter recommandation ET NLP depuis la meme table, contrairement a des datasets comme MovieLens qui n'ont pas de texte d'avis exploitable.
- Categorie cosmetique choisie pour son vocabulaire riche et concret (texture, odeur, irritation, efficacite) — plus parlant en NLP qu'une categorie technique, et plus intuitif a presenter que la categorie complete (23,9M avis, injouable sur un poste local).
- **Echantillonnage en streaming** : le fichier brut complet de la categorie fait ~311 Mo ; plutot que de le telecharger integralement, le pipeline lit le flux HTTP ligne par ligne et s'arrete des que 200 000 avis sont collectes (~93 Mo reellement transferees). Les metadonnees produit sont ensuite filtrees pour ne garder que les produits references dans l'echantillon.

3.2 Dictionnaire des colonnes — table reviews (avis)

Colonne	Type	Description metier
user_id	texte	Identifiant anonymise du client — cle du collaborative filtering
parent_asin	texte	Identifiant produit (regroupe les variantes couleur/taille d'un meme produit)
rating	1 a 5	Note laissee par le client
title	texte	Titre court redige par le client
text	texte	Corps de l'avis — matiere premiere du pipeline NLP
timestamp	entier	Date de publication (ms Unix) — sert au split train/test chronologique
helpful_vote	entier	Nombre de votes « utile » recus par l'avis
verified_purchase	bool	Achat verifie ou non (indicateur de fiabilite du signal)

3.3 Dictionnaire des colonnes — table items (catalogue produit)

Colonne	Description metier
parent_asin	Identifiant produit (cle primaire)
main_category	Categorie principale Amazon (ex. « All Beauty »)
title	Nom du produit
average_rating / rating_number	Note moyenne et nombre total d'avis, agreges sur l'ensemble d'Amazon (pas seulement l'echantillon)
price	Prix affiche (peut etre absent)
store	Marque / vendeur

Colonne	Description metier
categories	Sous-categories (arborescence), stockees en JSON texte
description / features	Texte descriptif et points cle product — utilises par le moteur content-based

4. Architecture du pipeline

SQL → Python → NLP → Recommendation → Evaluation → Dashboard

Toutes les étapes écrivent dans une seule base **SQLite** (data/portfolio.db), ce qui permet à chaque module de lire les sorties du précédent sans dépendance directe au code — une table SQL est un contrat plus stable qu'un objet Python en mémoire.

Etape	Script	Table(s) produite(s)
1. Ingestion	src/download_data.py	data/raw/*.jsonl
2. Chargement SQL	src/load_to_sql.py	reviews, items
3. NLP	src/nlp_pipeline.py	reviews_enriched, topics
4. Recommendation (CF)	src/recommend.py	train_reviews, test_reviews, item_similarity_top
5. Recommendation (content)	src/content_based.py	item_content_similarity_top
6. Evaluation	src/evaluate.py, src/evaluate_content.py	evaluation_results, evaluation_results_content
7. Dashboard	dashboard/app.py	(lecture seule, aucun calcul lourd)

5. Etape SQL — Ingestion et controle qualite

Objectif technique : simuler l'entrepot de donnees qu'aurait deja une equipe Data en entreprise — des avis bruts (JSON) transformes en tables structurees et interrogeables, avec un controle qualite systematique avant tout traitement en aval.

5.1 Controles effectues

Controle	Resultat
Avis avec texte vide	8 sur 50 000 (echantillon initial), nettoyes
Doublons exacts (meme user/produit/texte)	124 a 132 selon l'echantillon, nettoyes
Produits notes sans fiche produit (orphelins)	0 — coherence totale entre reviews et items
Total avis apres nettoyage NLP	199 119 / 200 000 (99,6% conservees)

5.2 Distribution des notes et sparsite

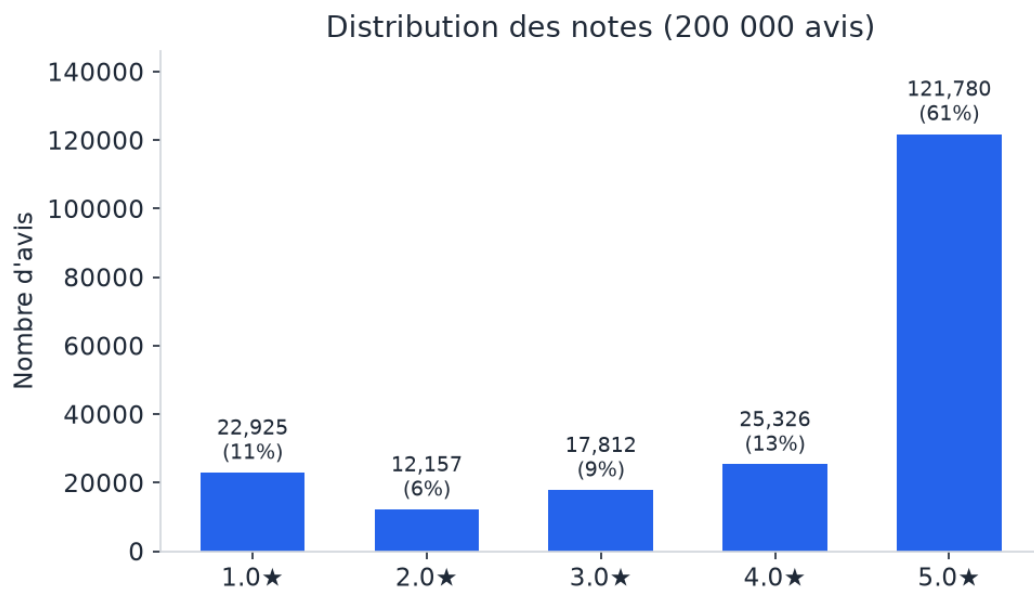


Figure 1 — Distribution des notes sur l'echantillon complet (200 000 avis).

Lecture metier : 61% des avis sont notes 5/5. C'est un biais tres classique en e-commerce (les clients satisfaits ecrivent proportionnellement plus d'avis, ou seuls les achats reussis declenchent un avis spontane). Consquence directe : un systeme naif « on recommande les mieux notes » donnerait une fausse impression de performance, d'ou la necessite d'une evaluation rigoureuse (section 9) plutot que de se fier aux notes brutes.

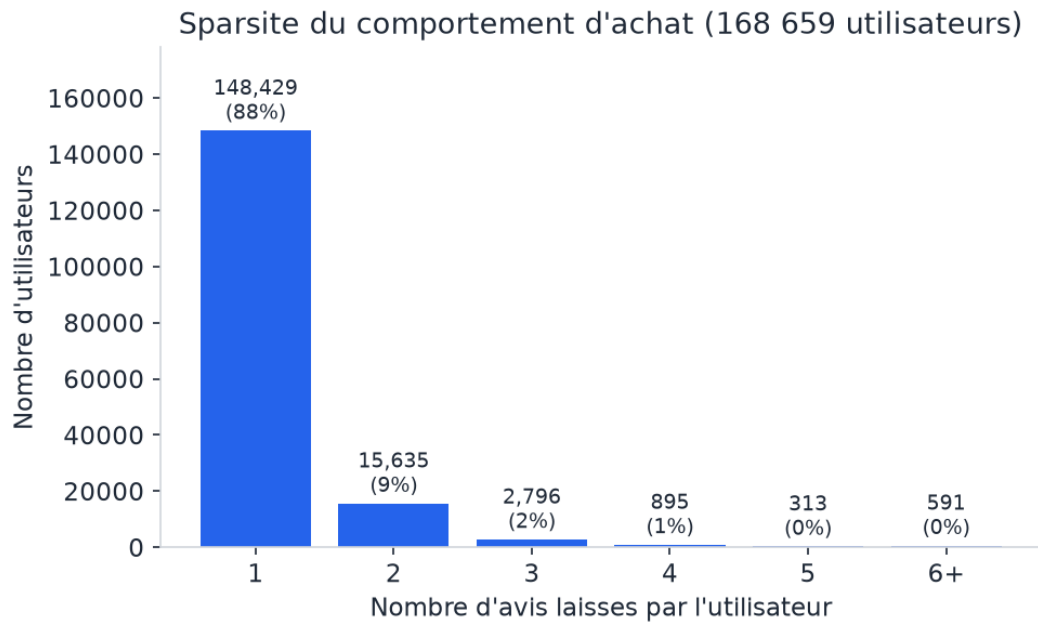


Figure 2 — Nombre d'avis par utilisateur : 88% des clients n'ont achete/note qu'une seule fois.

Impact direct sur la suite du projet : cette figure est la cle de lecture de tout le projet. La matrice user-item est extremement sparse (168 659 utilisateurs, 60 726 produits, 200 000 avis, densite $\approx 0,002\%$). Cette observation, faite des l'etape SQL, a directement motive l'architecture hybride CF + content-based presentee dans les sections 7 et 8.

6. Etape NLP — Analyse des avis clients

Objectif metier : transformer le texte libre en signaux exploitables pour l'equipe Qualite — sentiment par avis et themes recurrents par produit — sans lecture manuelle.

6.1 Nettoyage du texte

Suppression HTML residuel, URLs, caracteres speciaux, passage en minuscules, retrait des avis vides ou dupliques (cf. section 5.1).

6.2 Tokenisation et lemmatisation — pourquoi spaCy

Choix retenu : spaCy (en_core_web_sm) plutot que NLTK pour cette etape. **Justification** : spaCy traite le texte par lots (nlp.pipe) ce qui le rend nettement plus rapide sur 200 000 documents, et sa lemmatisation est basee sur un modele statistique plus precis que les regles de stemming classiques de NLTK (ex. « loved / loving / loves » → « love » correctement, y compris pour les formes irregulieres). NLTK reste utilise ailleurs dans le pipeline pour VADER (section 6.3), qui n'a pas d'equivalent aussi mature dans spaCy.

6.3 Analyse de sentiment — pourquoi VADER

Choix retenu : VADER (lexique NLTK), qui attribue a chaque avis un score continu de -1 (tres negatif) a +1 (tres positif), **independant de la note en etoiles**. **Justification** : VADER est specifiquement calibre pour du texte informel/avis clients (gere la negation « not good », l'intensite « !!! », les majuscules), ne necessite aucun entraînement (contrairement a un modele supervise, qui aurait demande des labels de sentiment que nous n'avions pas), et reste tres rapide sur 200 000 avis. Un modele transformer (ex. BERT sentiment) serait plus precis sur les tournures complexes mais représenterait un cout de calcul et de complexite disproportionne pour ce cas d'usage.

Repartition du sentiment (VADER, 199 119 avis)

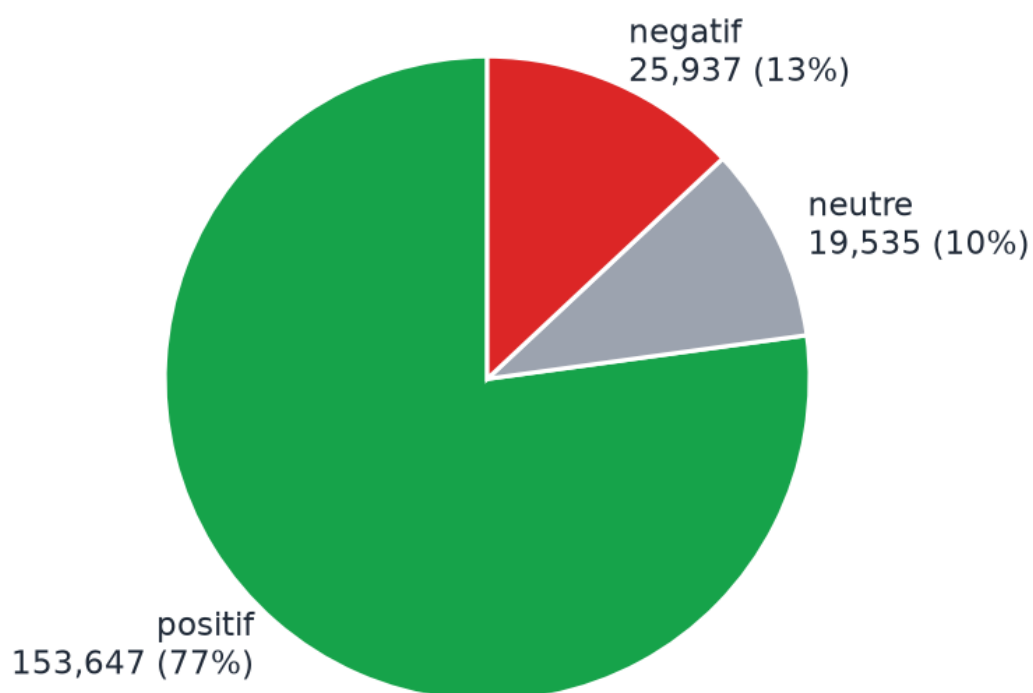


Figure 3 — Repartition du sentiment sur les 199 119 avis analyses.

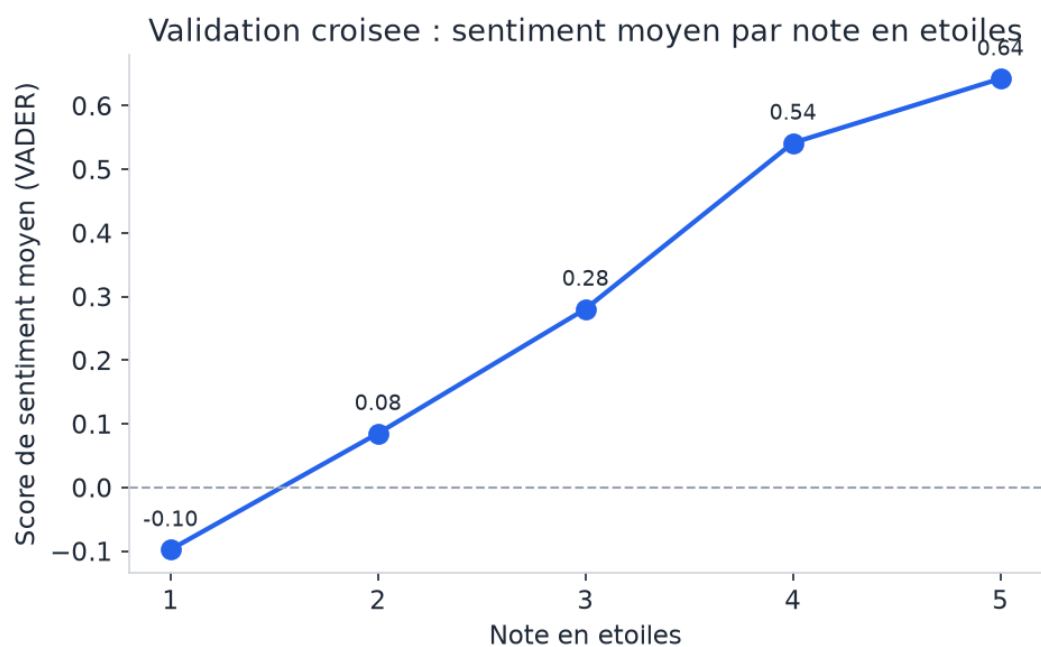


Figure 4 — Validation croisee : le score de sentiment moyen augmente de facon monotone avec la note en étoiles (-0,10 pour 1★ a +0,64 pour 5★), confirmant que VADER capte un vrai signal.

Limite observee. En cherchant des avis 5★ au texte pourtant tres negatif selon VADER, on trouve surtout des **faux positifs** de l'analyse lexicale : « Didn't expect much but got a nice beautiful surprise! » est scoree negativement a cause de « didn't expect », alors que l'avis est tres positif. C'est une limite connue des methodes a base de lexique (pas de vraie comprehension du contexte/ironie) — assumee et documentee plutot que cachee.

6.4 Topic modeling — pourquoi NMF

Choix retenu : NMF (Non-negative Matrix Factorization) sur une matrice TF-IDF (unigrammes + bigrammes), plutot que LDA (Latent Dirichlet Allocation). **Justification :** sur des textes courts comme des avis clients, NMF produit generalement des themes plus interpretables et plus stables que LDA (qui est concu pour des documents plus longs et repose sur une modelisation probabiliste plus couteuse). NMF est egalement deterministe a graine fixee, ce qui garantit la reproductibilite des resultats presentes ici. 8 themes ont ete retenus par iteration manuelle (4 initialement testes donnaient des groupes trop generiques ; l'ajout de stopwords specifiques au domaine — « buy », « use », « product »... — et des bigrammes a nettement ameliore la distinctivite des themes, par exemple en faisant emerger « smell good » ou « waste money » plutot que des mots isoles.

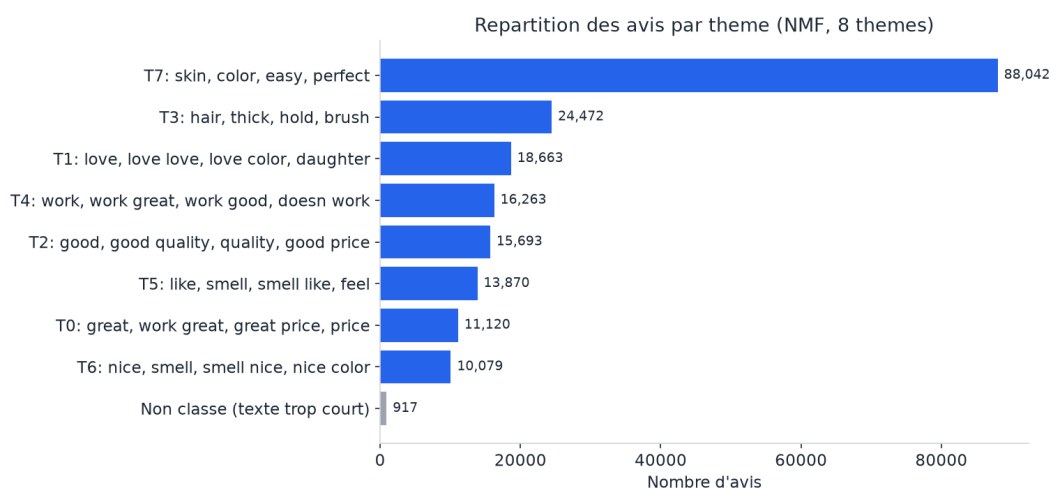


Figure 5 — Repartition des 199 119 avis sur les 8 themes identifies.

Limite observee. Le theme T7 (skin, color, easy, perfect...) regroupe a lui seul 44% des avis : c'est un theme « fourre-tout » assez generique, signe que 8 themes reste un compromis grossier pour un catalogue aussi varie (soins de la peau, cheveux, maquillage, ongles...). Segmenter le topic modeling par sous-categorie produit ameliorerait la finesse — piste d'amelioration documentee en section 12.

7. Moteur de recommandation — Collaborative Filtering

7.1 Pourquoi item-based plutôt que user-based

Avec 168 659 utilisateurs dont 88% n'ont qu'un seul avis, un collaborative filtering *user-based* (comparer des profils utilisateurs entre eux) est quasi inutilisable : il faut des utilisateurs aux profils suffisamment remplis pour être comparés. Le choix *item-based* (comparer des produits entre eux via les patterns de notation communs) est plus robuste ici, et c'est littéralement le mécanisme derrière le « customers who bought this also bought » d'Amazon.

7.2 Filtrage k-core — une décision expérimentale, pas arbitraire

La matrice user-item brute étant trop sparse pour calculer une similarité fiable, un filtrage **k-core** (ne garder que les utilisateurs/produits avec au moins N avis) est nécessaire. Plutôt que de choisir N arbitrairement, **4 configurations ont été testées** :

Configuration	Utilisateurs retenus	Lift Recall@20 vs popularité
Seuil=2, similarité brute	8 796	-53,6%
Seuil=2, similarité ajustée (centrée)	8 796	-83,4%
Seuil=5	1 (effondrement)	non calculable
Seuil=3, similarité ajustée (centrée)	606	+185,7%
Seuil=3, similarité brute (choix final)	606	+428,6%

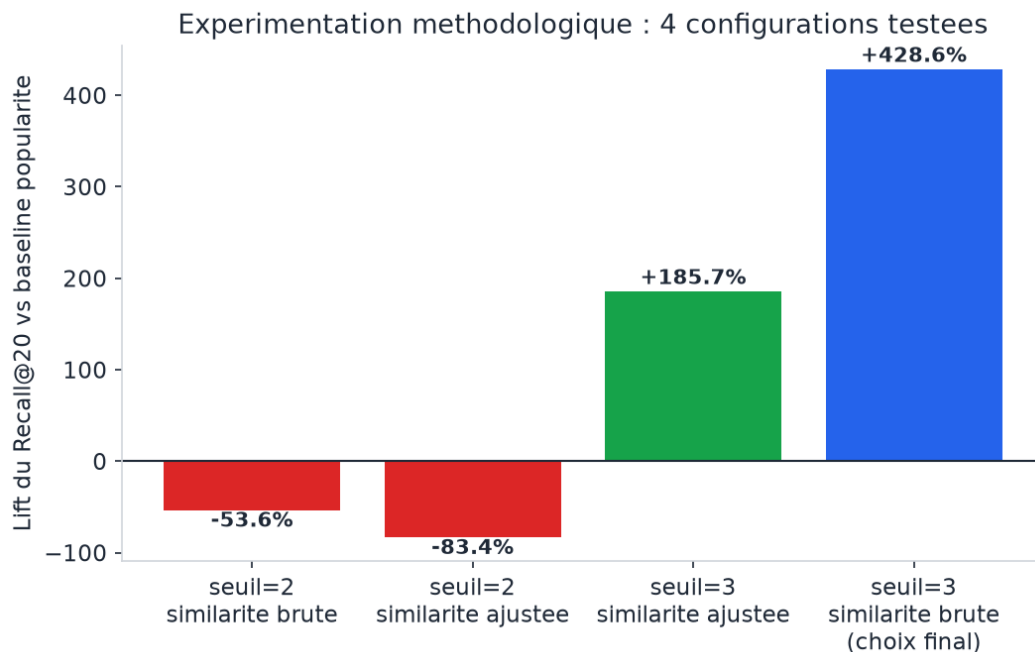


Figure 6 — Lift du Recall@20 selon la configuration testée.

Recit de la démarche (important pour comprendre le raisonnement, pas seulement le résultat) :

- **Hypothèse initiale** : au seuil=2, le CF fait moins bien que la simple popularité (-53,6%). Hypothèse formulée : le biais « 61% des avis sont notes 5★ » (section 5.2) pollue la similarité cosinus brute, qui capterait surtout « qui a noté ce produit » plutôt que « qui l'a vraiment préféré ».

- **Test de l'hypothese** : implementation d'une similarite cosinus *ajustee* (Sarwar et al., 2001) qui centre chaque note sur la moyenne de l'utilisateur avant de comparer les produits. Resultat : **encore pire** (-83,4%). L'hypothese est rejetee.
- **Cause racine identifiee** : au seuil=2, la plupart des utilisateurs n'ont que 2 avis — centrer sur une moyenne de 2 points est du bruit pur, pas un signal. Le vrai probleme est la **sparsite**, pas le biais de notation.
- **Correction** : relever le seuil. A seuil=5, le filtrage iteratif (utilisateur ET produit doivent tous deux atteindre le seuil) fait s'effondrer le jeu de donnees a 1 seul utilisateur — ce dataset de niche n'a pas assez d'acheteurs recurrents pour un seuil aussi eleve.
- **Validation finale** : au seuil=3 (compromis), la similarite *ajustee* devient positive (+185,7%) mais la similarite *brute* fait **nettement mieux** (+428,6%) — confirmant que le centrage reste bruite meme a ce seuil, avec seulement 3 points par utilisateur en moyenne.

Point cle. Cette experimentation illustre une methode plus qu'un resultat : une hypothese a ete formulee, testee, **rejetee** sur preuve empirique, et la vraie cause a ete identifiee et corrgee. C'est ce raisonnement — plus que le chiffre final — qui constitue la valeur methodologique de cette section.

7.3 Protocole final retenu

- **Filtrage** : k-core iteratif, seuil = 3 avis minimum (utilisateurs et produits).
- **Similarite** : cosinus brute entre colonnes-produits de la matrice user-item (train).
- **Split train/test** : leave-one-out chronologique — le dernier avis de chaque utilisateur (par date) est retire pour servir de test, le reste sert a l'entrainement.
- **Scoring** : moyenne ponderee des notes de l'utilisateur sur les produits similaires, normalisee par la somme des similarites absolues (evite qu'un produit avec beaucoup de voisins faibles domine un produit avec peu de voisins mais tres similaires).

8. Moteur de recommandation — Content-Based (fallback)

8.1 Pourquoi ce module est nécessaire

Le collaborative filtering (section 7) ne couvre que **606 utilisateurs sur 168 659** (0,4%) : le seuil $k\text{-core}=3$ exclut structurellement tout utilisateur avec moins de 3 avis, soit **164 064 utilisateurs** (97%). Un système de recommandation en production ne peut pas se permettre de ne servir que 0,4% de sa base.

8.2 Principe et choix technique

Le content-based recommande des produits en se basant sur leurs **caractéristiques** (titre, description, points clés, catégories, marque) plutôt que sur le comportement croisé d'autres utilisateurs. **Avantage clé** : il ne nécessite *aucun historique multiple* — un seul avis positif suffit comme point d'ancrage pour générer une recommandation.

Choix technique : vectorisation TF-IDF (titre + description + features + catégories + marque, 20 000 termes max) puis similarité cosinus entre les **60 725 produits** du catalogue complet, calculée par lots de 1 000 produits pour rester tractable en mémoire (une matrice dense complète $60\,725 \times 60\,725$ représenterait ~14 Go, inutile ici puisque seuls les 20 plus proches voisins de chaque produit sont conservés).

8.3 Exemple concret

Pour le produit « 4 Point Eyebrow Pen, Micro Ink Tat Brow Pen Waterproof... », les 10 produits les plus similaires par contenu sont tous des crayons/stylos sourcils ou produits de microblading — une cohérence sémantique immédiatement lisible, contrairement aux recommandations du CF qui reflètent des corrélations statistiques moins intuitives à interpréter visuellement.

9. Evaluation — Methodologie et resultats

9.1 Metriques : Precision@K et Recall@K

Pour chaque utilisateur du jeu de test, on genere une liste de K produits recommandes et on verifie si le produit reellement note ensuite (le « held-out » du split train/test) y figure.

- **Precision@K** = parmi les K produits recommandes, quelle proportion correspond a ce que l'utilisateur a reellement note ensuite ?
- **Recall@K** = le produit reellement note ensuite fait-il partie des K recommandes ?

Precision methodologique. Notre protocole leave-one-out ne retient qu'un **seul avis test par utilisateur**. Consquence mathematique : Recall@K devient equivalent a un taux de succes (« hit rate », 0 ou 1 par utilisateur), et Precision@K = ce taux divise par K. Cette nuance est documentee ici pour ne pas laisser croire a une Precision@K au sens plein du terme (plusieurs items pertinents par utilisateur).

9.2 Baseline de comparaison

Un modele naif « produits les plus populaires » (memes recommandations pour tout le monde, sans aucune personnalisation) sert de reference obligatoire : si un modele plus complexe ne le bat pas, il n'apporte aucune valeur.

9.3 Resultats — Collaborative Filtering

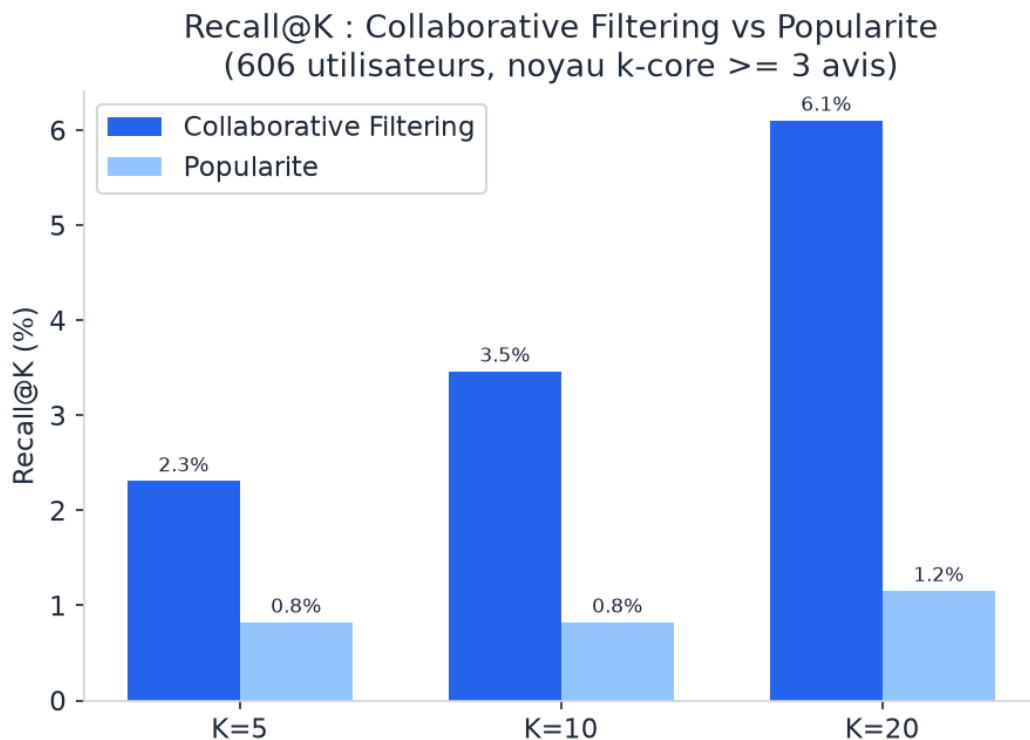


Figure 7 — Recall@K : Collaborative Filtering vs baseline popularite (606 utilisateurs).

K	CF Recall	Popularite Recall	Lift
5	2,31%	0,83%	+180,0%
10	3,47%	0,83%	+320,0%
20	6,11%	1,16%	+428,6%

9.4 Resultats — Content-based (fallback)

Evalue sur la cohorte « **exactement 2 avis** » (15 631 utilisateurs) — le segment precisement exclu du CF (seuil k-core ≥ 3), donc le trou que ce module cherche a combler. Protocole identique : 1er avis = seed, 2e avis (le plus recent) = cible a retrouver.

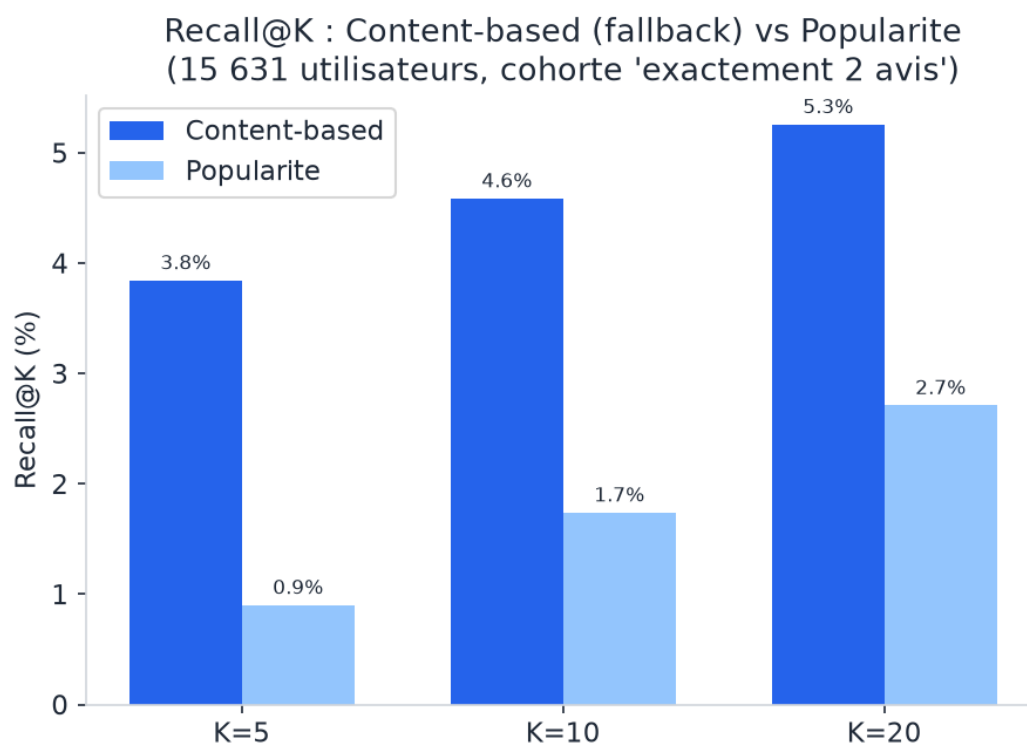


Figure 8 — Recall@K : Content-based vs baseline popularite (15 631 utilisateurs).

K	Content Recall	Popularite Recall	Lift
5	3,84%	0,90%	+326,2%
10	4,59%	1,74%	+163,6%
20	5,26%	2,71%	+93,9%

A noter : l'echantillon d'evaluation du content-based (15 631 utilisateurs) est **25 fois plus grand** que celui du CF (606), ce qui rend ce resultat statistiquement plus robuste malgre un lift en pourcentage plus modere a K eleve.

9.5 Couverture globale du systeme hybride

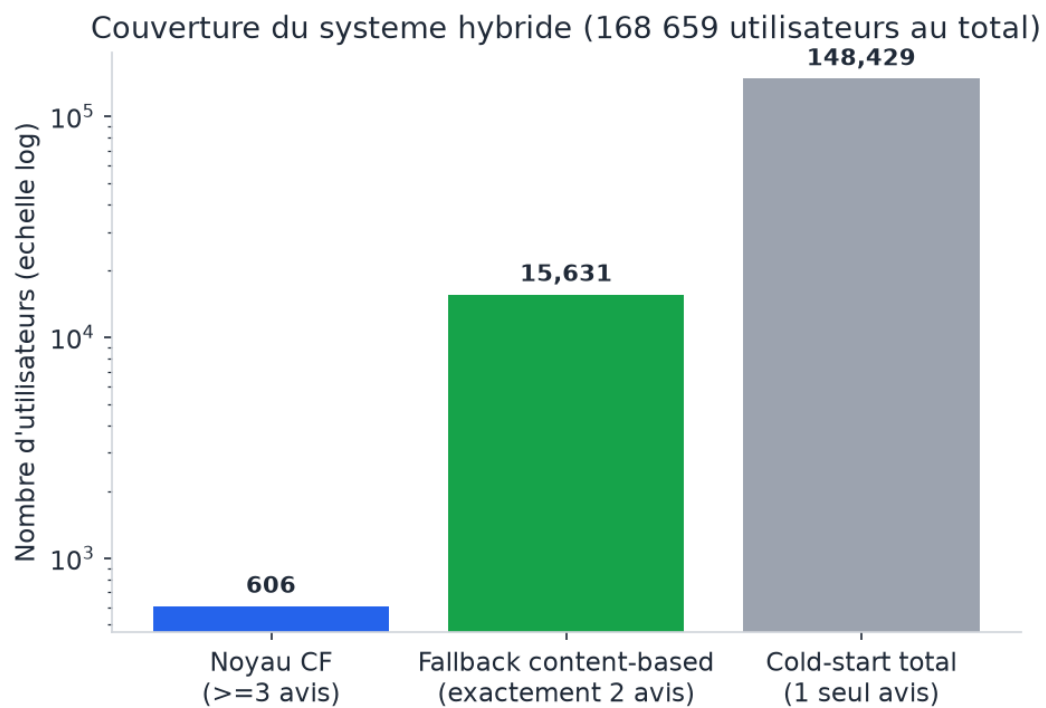


Figure 9 — Couverture du systeme hybride par segment d'utilisateurs (echelle logarithmique).

Le systeme combine couvre **16 237 utilisateurs** (606 + 15 631) avec une methode evaluee et validee, soit **9,6%** de la base totale. Les 148 429 utilisateurs restants (avis unique) recevraient la baseline popularite en production, en attendant un second achat.

10. Dashboard Streamlit — documentation des visuels

Le dashboard (dashboard/app.py) rend l'ensemble de ce travail consultable sans toucher au code, via 4 onglets lisant exclusivement la base SQLite precalculee (aucun calcul lourd au chargement de la page).

10.1 Onglet « Vue d'ensemble »

- 3 indicateurs cles (KPI) : nombre d'avis, note moyenne, nombre de produits.
- Graphique en barres de la distribution des notes (identique a la Figure 1) — situe le biais de notation des la premiere vue.
- Graphique en secteurs de la repartition du sentiment (identique a la Figure 3) — vue « Qualite » du catalogue en un coup d'oeil.

10.2 Onglet « Analyse par produit »

- Selecteur limite aux 200 produits les plus commentes (evite une liste de 60 725 entrees illisible).
- Note moyenne et nombre d'avis du produit selectionne.
- Graphique du sentiment specifique a ce produit, et des themes dominants parmi ses avis (reutilise la table topics de la section 6.4).
- 3 avis les plus positifs et 3 les plus negatifs (tries par sentiment_score) — permet une lecture qualitative rapide sans parcourir tous les avis du produit.

10.3 Onglet « Recommandation »

- **Section 1 (CF)** : selecteur parmi les 100 utilisateurs les plus actifs du noyau CF, affichage des produits deja notes puis des 10 recommandations avec leur score de similarite.
- **Section 2 (content-based)** : selecteur de produit (parmi les 300 plus commentes), affichage des 10 produits les plus similaires par contenu — demonstration independante de tout historique utilisateur.

10.4 Onglet « Evaluation »

- Tableau et graphique Precision@K/Recall@K pour le CF (section 9.3).
- Tableau et graphique Precision@K/Recall@K pour le content-based (section 9.4).
- 3 indicateurs de couverture globale (section 9.5) et un encart de limites methodologiques.

11. Rigueur des donnees : un bug detecte et corrige

Lors de la preparation de ce rapport, une verification de la correlation sentiment/note (section 6.3) a revele un resultat aberrant : le score de sentiment moyen etait quasi identique ($\sim 0,48$) pour toutes les notes de 1 a 5, contredisant la validation initiale realisee sur l'echantillon de 50 000 avis.

Diagnostic : la table `reviews_enriched` referençait des identifiants d'avis (`review_id` 50 001 a 250 000) qui ne correspondaient plus a la table `reviews` courante (`review_id` 1 a 200 000). Cause racine : en SQLite, `DELETE FROM table` ne reinitialise pas le compteur `AUTOINCREMENT` (seul `DROP TABLE` le fait). Le rechargement en deux temps de l'echantillon (50k puis 200k avis) a donc decalé les identifiants, puis un `DROP TABLE` introduit plus tard (pour ajouter les colonnes `description/features`) les a de nouveau reinitialises sans mettre a jour la table enrichie correspondante — rendant la jointure partiellement aleatoire.

Correction : le pipeline NLP a ete relance sur la table `reviews` courante, restaurant l'alignement (199 119/199 119 avis correctement joints) et la correlation attendue (sentiment -0,10 pour 1★ a +0,64 pour 5★, cf. Figure 4). Le script `load_to_sql.py` utilise desormais systematiquement `DROP TABLE` pour eviter toute recurrence.

Pourquoi documenter ce bug. Ce type de bug (silencieux, sans erreur ni crash — juste des chiffres plausibles mais faux) est particulierement dangereux en production. Il est documente ici deliberelement : la rigueur consiste a verifier ses propres resultats avant de les publier, pas seulement a faire fonctionner le code.

12. Limites assumees et pistes d'amelioration

Limite	Detail	Piste d'amelioration
Cold-start total non couvert	148 429 utilisateurs (88%) avec un seul avis, sans second point de donnee pour valider une recommandation	Backoff vers popularite par sous-categorie ; collecte de signaux implicites (vues, clics) en complement des avis
Echantillon d'evaluation CF modeste	606 utilisateurs — le lift mesure est directionnel, sans test de significativite statistique formel	Bootstrap ou intervalle de confiance sur le Recall@K ; etendre a d'autres categories pour verifier la generalisation
Sentiment lexical (VADER)	Faux positifs/negatifs sur l'ironie, les negations complexes ou les phrases mixtes (cf. section 6.3)	Modele transformer fine-tune sur avis clients (ex. DistilBERT sentiment) pour les cas ambigus
Topic modeling peu granulaire	Un theme (T7) concentre 44% des avis, signe d'une segmentation trop grossiere pour un catalogue varie	Topic modeling par sous-categorie produit plutot que sur le catalogue entier
Catalogue de niche	All Beauty est une petite categorie (par rapport a Beauty_and_Personal_Care, 23,9M avis) — le taux de rachat y est structurellement bas	Reproduire le pipeline sur une categorie a plus fort taux de rechat (ex. produits consommables) pour comparer la couverture du CF

13. Annexe technique

13.1 Stack technique

Outil	Rôle dans le pipeline
SQLite	Base de données unique, partagée par tous les modules
pandas / scipy (sparse)	Manipulation de données, matrice user-item creuse
spaCy (en_core_web_sm)	Tokenisation et lemmatisation
NLTK (VADER, stopwords)	Analyse de sentiment lexicale
scikit-learn (TF-IDF, NMF, cosine_similarity)	Topic modeling et similarité (CF + content-based)
Streamlit + Plotly	Dashboard interactif
Hugging Face (streaming HTTP)	Ingestion des données sans téléchargement complet

13.2 Structure du repository

src/ (scripts du pipeline) · sql/schema.sql (schema SQLite) · dashboard/app.py (application Streamlit) · reports/ (generation de ce rapport) · data/ (données générées localement, non versionnées).

13.3 Execution complete

```
python src/download_data.py && python src/load_to_sql.py && python  
src/nlp_pipeline.py && python src/recommend.py && python src/content_based.py  
&& python src/evaluate.py && python src/evaluate_content.py && streamlit run  
dashboard/app.py
```